



PKI-VAULT

Infrastructure a Cles Publiques a Trois Niveaux

Module : Cryptographie et Blockchain

Filiere : Master MMSD (Mathematiques, Modelisation et Sciences des Donnees)

Etablissement : Faculte des Sciences et Techniques, Tanger

Encadrante : Pr. LECHHAB OUADRASSI Nihad

Co-superviseur : Pr. AZMANI Abdellah

Realise par : **Akiyaf Achraf**

Environnement : Kali Linux · OpenSSL · Python 3 · Flask

Juin 2025

[\[GitHub \]](#) achrafaky/pki-vault-da-vinci

Resume

Ce projet implemente une **Infrastructure a Cles Publiques (PKI) a trois niveaux** nommee **PKI-VAULT**, deploye sous Kali Linux. La chaine de confiance comprend une **Root CA** (RSA 4096 bits, 20 ans), une **Intermediate CA** (RSA 4096, 10 ans) et des **certificats Leaf** (RSA 2048 / ECDSA P-384, 365 jours). L'outil principal est **OpenSSL** pour toutes les operations cryptographiques, complete par une **application web Flask** avec tableau de bord interactif. Ce rapport documente les etapes de creation, la configuration OpenSSL, les scripts Bash, et une phase de tests complete de la generation a la revocation.

Mots-cles : PKI, X.509, Root CA, Intermediate CA, Leaf, CSR, CRL, OpenSSL, RSA, ECDSA, Flask, revocation.

Table des matieres

1	Introduction	3
2	Architecture de la PKI	3
2.1	Hierarchie a trois niveaux	3
2.2	Justification des choix cryptographiques	4
2.3	Arborescence du projet	4
3	Phase 1 — Creation et Configuration du Projet	5
3.1	Installation des prerequis	5
3.2	Initialisation de l'environnement Python	5
3.3	Configuration OpenSSL	5
3.3.1	Root CA — config/root.cnf	6
3.3.2	Intermediate CA — config/inter.cnf	6
3.4	Scripts Bash automatises	7
3.5	Application Flask — Routes	7
3.6	Script de synchronisation base de donnees (sync_db.py)	7
4	Phase de Tests — Etapes 1 a 12	8
4.1	Etape 1 — Activation de l'environnement virtuel	9
4.2	Etape 2 — Reinitialisation de la base de donnees	9

4.3	Etape 3 — Generation CSR et signature : web1.local (RSA 2048)	10
4.4	Etape 4 — Generation CSR et signature : web2.local (RSA 2048)	12
4.5	Etape 5 — Generation CSR et signature : secure.local (ECDSA P-384) . . .	13
4.6	Etape 7 — Synchronisation base de donnees et verification	14
4.7	Etape 8 — Revocation de web1.local	15
4.8	Etape 9 — Verification post-revocation	16
4.9	Etape 10a — Signature via l'interface web	16
4.10	Etape 10b — Verification via l'interface web	17
4.11	Etape 11 — Tests supplementaires : API REST et downloads	18
4.12	Etape 12 — Etat final : tableau de bord PKI-VAULT	19
4.13	Tests terminaux avances — Synthese	19
4.13.1	Verification de 3 certificats simultanement	20
4.13.2	Revocation + verification en chaine	20
5	Tableau de Bord — Dashboard PKI-VAULT	20
5.1	Vue principale	21
5.2	Liste des certificats	21
5.3	Fonctionnalites avancees	22
6	Resultats et Analyse	22
6.1	Tableau de synthese des tests	23
6.2	Points forts du projet	23
6.3	Pistes d'amelioration	23
7	Conclusion	24
	Annexes	25

1. Introduction

Une **Infrastructure a Cles Publiques** (PKI) est un ensemble de mecanismes cryptographiques permettant de gerer des certificats numeriques X.509. Elle garantit l'authenticite, la confidentialite et l'integrite des communications numeriques.

Le projet **PKI-VAULT** met en place une PKI complete et fonctionnelle a trois niveaux, avec les objectifs suivants :

1. Comprendre et implementer la hierarchie et la chaine de confiance dans une PKI.
2. Generer, signer et distribuer des certificats numeriques (RSA et ECDSA).
3. Securiser les cles privees (permissions strictes, stockage isole).
4. Implementer la gestion des CSR et des CRL.
5. Proposer une interface web Flask pour toutes les operations PKI.
6. Verifier la validite et la chaine de confiance des certificats.

Code source disponible sur GitHub

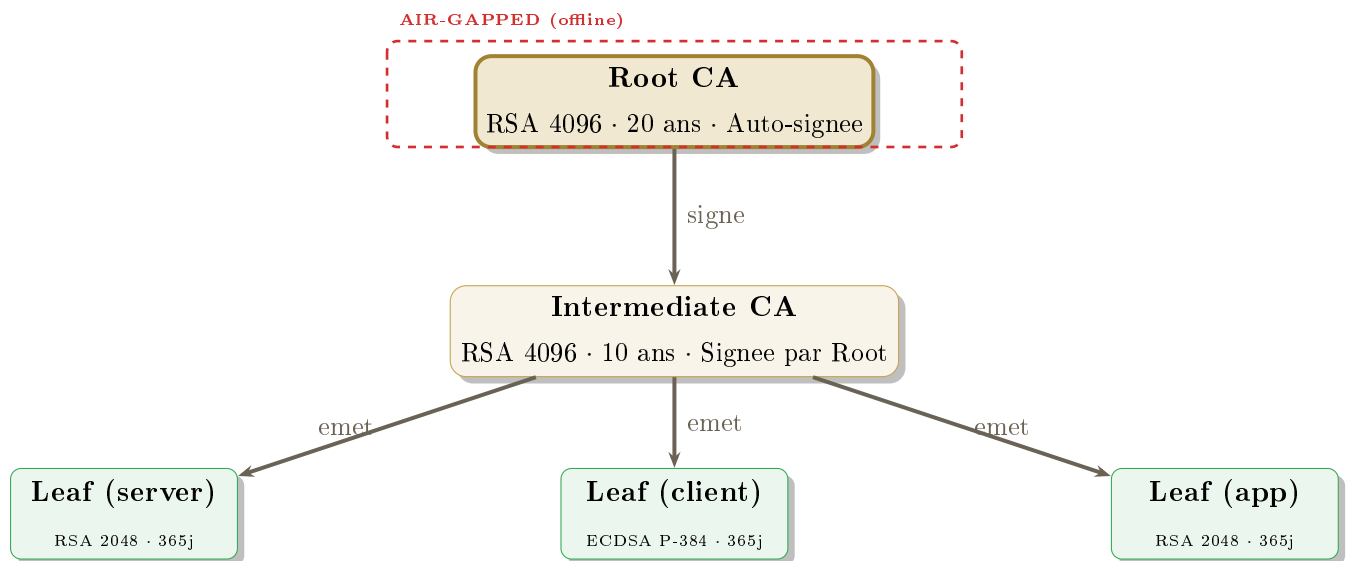
<https://github.com/achrafaky/pki-vault-da-vinci>

Environnement technique

Systeme :	Kali Linux (utilisateur : axraks08)
Langage :	Python 3, Bash
Bibliotheques :	Flask, cryptography, python-dotenv
Outil PKI :	OpenSSL (version recente)
Base de donnees :	SQLite (pki_vault.db)
Interface :	Tableau de bord Flask — port 5000

2. Architecture de la PKI

2.1 Hierarchie a trois niveaux



Principe fondamental : la Root CA reste *hors ligne* (air-gapped) en production. Seule l'Intermediate CA est operationnelle en permanence.

2.2 Justification des choix cryptographiques

Entite	Algo.	Taille	Duree	Justification
Root CA	RSA	4096 bits	20 ans	Securite maximale, rarement utilisee
Intermediate CA	RSA	4096 bits	10 ans	Protection Root + souplesse
Leaf (serveur)	RSA	2048 bits	365 j	Standard TLS actuel
Leaf (securise)	ECDSA	P-384	365 j	Performance + securite post-2030
Hash	—	SHA-256	—	SHA-1 explicitement rejete

TABLE 1 – Parametres cryptographiques de la PKI-VAULT

2.3 Arborescence du projet

[bash] Structure des fichiers — pki-vault-da-vinci/

```

pki-vault-da-vinci/
+-- .env                # Variables d'environnement (mots de passe CA)
+-- requirements.txt    # Dependances Python
+-- run.py              # Point d'entree Flask
+-- Makefile            # Automatisation des taches
+-- sync_db.py          # Synchronisation index.txt -> SQLite
+-- pki_vault.db        # Base de donnees SQLite
+-- config/

```

```
|   +-- root.cnf           # Configuration OpenSSL Root CA
|   +-- inter.cnf         # Configuration OpenSSL Intermediate CA
+-- scripts/
|   +-- init_ca.sh        # Initialisation PKI (Root + Inter + CRL)
|   +-- gen_csr.sh        # Generation cle + CSR
|   +-- sign_cert.sh      # Signature certificat Leaf
|   +-- revoke.sh         # Revocation + mise a jour CRL
|   +-- verify.sh         # Verification certificat (5 checks)
|   +-- demo.sh           # Scenario de demonstration complet
+-- pki/
|   +-- root/{private,certs,crl,db,newcerts}/
|   +-- intermediate/{private,certs,crl,db,newcerts,csr}/
|   +-- leaf/{private,certs,csr}/
+-- app/
|   +-- app.py             # Routes Flask (dashboard, generate, sign...)
|   +-- models.py         # Modeles SQLite + statistiques
|   +-- crypto_utils.py   # Couche cryptographique
|   +-- templates/        # Pages HTML (Jinja2)
|   +-- static/{css,js}/  # Style Da Vinci Light 3D
```

3. Phase 1 — Creation et Configuration du Projet

3.1 Installation des prerequis

[bash] Installation des outils systeme

```
sudo apt update && sudo apt install -y openssl python3 python3-pip \
python3-venv git tree
```

3.2 Initialisation de l'environnement Python

[bash] Creation du projet et environnement virtuel

```
cd ~
mkdir -p pki-vault-da-vinci && cd pki-vault-da-vinci
python3 -m venv venv && source venv/bin/activate
pip install flask cryptography python-dotenv
```

3.3 Configuration OpenSSL

3.3.1 Root CA — config/root.cnf

[bash] Extrait config/root.cnf

```
[ CA_default ]
default_days = 3650      # 10 ans pour les certificats signes
default_md   = sha256    # SHA-256 obligatoire
rand_serial  = yes       # Numeros de serie aleatoires

[ policy_strict ]
countryName   = match    # C= doit correspondre exactement
organizationName = match # O= doit correspondre exactement
commonName    = supplied # CN= fourni par le demandeur

[ v3_ca ]
basicConstraints = critical, CA:true, pathlen:1
keyUsage = critical, digitalSignature, cRLSign, keyCertSign
```

3.3.2 Intermediate CA — config/inter.cnf

[bash] Extrait config/inter.cnf

```
[ CA_default ]
default_days = 365      # 1 an pour les certificats Leaf
copy_extensions = copy  # Copier les SAN depuis la CSR

[ server_cert ]
basicConstraints = CA:FALSE
keyUsage = critical, digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth

[ client_cert ]
basicConstraints = CA:FALSE
keyUsage = critical, digitalSignature, nonRepudiation
extendedKeyUsage = clientAuth
```


3.4 Scripts Bash automatisés

Script	Fonction
<code>init_ca.sh</code>	Cree Root CA, Intermediate CA, CRL initiales, verifie la chaine
<code>gen_csr.sh</code>	Genere cle (RSA ou ECDSA) + CSR avec SAN
<code>sign_cert.sh</code>	Signe une CSR avec l'Intermediate CA
<code>revoke.sh</code>	Revoque un certificat + regenere la CRL
<code>verify.sh</code>	Verifie format, dates, chaine, CRL, algo (score /5)
<code>demo.sh</code>	Scenario complet : init, generation, revocation

TABLE 2 – Description des scripts Bash

3.5 Application Flask — Routes

Route	Methode	Fonctionnalite
<code>/</code>	GET	Tableau de bord (KPI, graphiques, timeline)
<code>/generate</code>	GET/POST	Generation de CSR via formulaire web
<code>/sign</code>	GET/POST	Signature de CSR uploadée
<code>/certificates</code>	GET	Liste des certificats avec filtres
<code>/revoke/<id></code>	POST	Revocation d'un certificat
<code>/verify</code>	GET/POST	Verification certificat (score /6)
<code>/viz</code>	GET	Vue 3D de la chaine PKI (Three.js)
<code>/api/stats</code>	GET	Statistiques JSON en temps reel
<code>/download/crl</code>	GET	Telechargement de la CRL
<code>/download/chain</code>	GET	Telechargement de la chaine de confiance
<code>/kill/<ca_type></code>	POST	Kill Switch — revocation massive

TABLE 3 – Routes de l'application Flask

3.6 Script de synchronisation base de donnees (sync_db.py)

Ce script lit le fichier `index.txt` d'OpenSSL et synchronise son etat avec la base SQLite utilisee par Flask.

[bash] Fonctionnement de sync_db.py

```
python3 sync_db.py
# Lit pki/intermediate/db/index.txt
# Parse chaque ligne : statut (V/R), date expiration, serial, CN
# Insere ou met a jour les entrees dans la table 'certificates'
# Resultat : "Base de donnees synchronisee"
```

```
(venv)-(axraks08@kali)-[~/pki-vault-da-vinci]
$ >....
    exp_date = parts[1]
    rev_date = parts[2] if parts[2] else None
    serial = parts[3]
    subject = parts[5]

    # Extraire CN
    cn_match = re.search(r'CN=([^,/]+)', subject)
    cn = cn_match.group(1) if cn_match else "unknown"

    # Déterminer le statut
    if status_flag == 'V':
        status = 'valid'
        revoked_at = None
        rev_reason = None
    elif status_flag == 'R':
        status = 'revoked'
        revoked_at = rev_date
        rev_reason = 'keyCompromise'
    else:
        status = 'expired'
        revoked_at = None
        rev_reason = None

    # Vérifier si le certificat existe déjà
    cursor.execute("SELECT id FROM certificates WHERE serial = ?", (serial,))
    if cursor.fetchone():
        continue

    # Chemin du certificat
    cert_path = Path(f"pki/leaf/certs/{cn.replace('.', '_')}_local.crt.pem")
    if not cert_path.exists():
        cert_path = Path(f"pki/leaf/certs/{cn}.crt.pem")
        if not cert_path.exists():
            cert_path = None

    # Insérer
    cursor.execute("""
        INSERT INTO certificates (cn, serial, status, cert path, expires at, revoked at, rev
reason)
        VALUES (?, ?, ?, ?, ?, ?, ?)
        """, (cn, serial, status, str(cert_path) if cert_path else None, exp_date, revoked_at, r
ev_reason))

    conn.commit()
    conn.close()
    print("✓ Base de données synchronisée")
EOF
```

FIGURE 1 – Code du script sync_db.py — parsing de index.txt et insertion en base SQLite

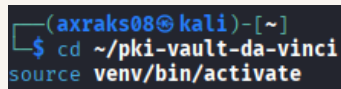
4. Phase de Tests — Etapes 1 a 12

4.1 Etape 1 — Activation de l'environnement virtuel

Etape 1 — Acces au repertoire et activation du virtualenv Python

[bash] Commandes — Etape 1

```
cd ~/pki-vault-da-vinci
source venv/bin/activate
```



```
(axraks08@kali)-[~]
$ cd ~/pki-vault-da-vinci
source venv/bin/activate
```

FIGURE 2 – Terminal Kali — activation du virtualenv pki-vault-da-vinci

Le prefixe (venv) confirme que l'environnement virtuel Python est actif. Toutes les bibliotheques (Flask, cryptography...) sont isolees dans cet environnement et n'interferent pas avec le systeme.

4.2 Etape 2 — Reinitialisation de la base de donnees

Etape 2 — Suppression et recreation de la base SQLite

[bash] Commandes — Etape 2

```
# Supprimer l'ancienne base de donnees
rm -f pki_vault.db

# Recreer la base vide avec les tables
python3 -c "
from app.models import init_db
init_db()
print('[OK] Base de donnees reinitialisee')
"
```

```
(venv)-(axraks08@kali)-[~/pki-vault-da-vinci]
$ # Supprimer l'ancienne base de données
rm -f pki_vault.db

# Recréer la base vide avec les tables
python3 -c "
from app.models import init_db
init_db()
print('✓ Base de données réinitialisée')
"
✓ Base de données réinitialisée
```

FIGURE 3 — Reinitialisation de pki_vault.db — tables certificates et audit_logs recrées

[OK] Base de donnees reinitialisee — la fonction `init_db()` execute les scripts SQL de creation des tables, assurant un etat initial coherent.

4.3 Etape 3 — Generation CSR et signature : web1.local (RSA 2048)

Etape 3 — Creation du premier certificat Leaf — RSA 2048 bits

[bash] Commandes — Etape 3

```
bash scripts/gen_csr.sh "web1.local" "" rsa 2048
bash scripts/sign_cert.sh "web1.local" server 365
```

```

(venv)-(axraks08@kali)-[~/pki-vault-da-vinci]
$ bash scripts/gen_csr.sh "web1.local" "" rsa 2048
bash scripts/sign_cert.sh "web1.local" server 365

— Génération : web1.local (rsa 2048) —
[✓] Clé RSA 2048 bits générée
[✓] CSR créée : /home/axraks08/pki-vault-da-vinci/pki/leaf/csr/web1_local.csr.pem
Clé : /home/axraks08/pki-vault-da-vinci/pki/leaf/private/web1_local.key.pem
CSR : /home/axraks08/pki-vault-da-vinci/pki/leaf/csr/web1_local.csr.pem

— Signature : web1.local (server : 365 jours) —
Using configuration from config/inter.cnf
Check that the request matches the signature
Signature ok
Certificate Details:
    Serial Number:
        4d:6e:fa:cb:ec:e1:c1:b8:5c:12:78:6a:18:31:1c:a5:c9:dc:6e:7e
    Validity
        Not Before: Jun 27 19:05:15 2026 GMT
        Not After : Jun 27 19:05:15 2027 GMT
    Subject:
        countryName           = MA
        stateOrProvinceName   = Tanger
        organizationName      = PKI Vault Enterprise
        commonName            = web1.local
    X509v3 extensions:
        X509v3 Basic Constraints:
            CA:FALSE
        X509v3 Key Usage: critical
            Digital Signature, Key Encipherment
        X509v3 Extended Key Usage:
            TLS Web Server Authentication
        X509v3 Subject Alternative Name:
            DNS:web1.local
Certificate is to be certified until Jun 27 19:05:15 2027 GMT (365 days)

Write out database with 1 new entries
Can't open "/home/axraks08/pki-vault-da-vinci/pki/leaf/certs/web1_local.crt.pem" for writing, Permission denied
400725FB457F0000:error:8000000D:system library:BIO_new_file:Permission denied:../crypto/bio/bss_file.c:67:calling fopen(/home/axraks08/pki-vault-da-vinci/pki/leaf/certs/web1_local.crt.pem, w)
400725FB457F0000:error:10080002:BIO routines:BIO_new_file:system lib:../crypto/bio/bss_file.c:77
:

```

FIGURE 4 – Generation CSR + signature de web1.local — RSA 2048 bits, 365 jours

Analyse du resultat :

- **[OK]** Cle RSA 2048 bits generee dans pki/leaf/private/
- **[OK]** CSR creee dans pki/leaf/csr/web1_local.csr.pem
- Certificat valide du **27 juin 2026** au **27 juin 2027** (365 jours)
- Extensions X.509v3 : CA:FALSE, TLS Web Server Authentication, SAN: DNS:web1.local

Erreur de permission — contexte

L'erreur BIO:new_file:Permission denied apparait car le fichier web1_local.crt.pem existait deja avec les droits 444 (lecture seule, depuis une session precedente). Le certificat a ete enregistre dans index.txt. La synchronisation via sync_db.py resout l'incoherence avec la base SQLite.

4.4 Etape 4 — Generation CSR et signature : web2.local (RSA 2048)

Etape 4 — Deuxieme certificat Leaf RSA — detection de doublon

[bash] Commandes — Etape 4

```
bash scripts/gen_csr.sh "web2.local" "" rsa 2048
bash scripts/sign_cert.sh "web2.local" server 365
```

```
(venv)-(axraks08@kali)-[~/pki-vault-da-vinci]
$ bash scripts/gen_csr.sh "web2.local" "" rsa 2048
bash scripts/sign_cert.sh "web2.local" server 365

— Génération : web2.local (rsa 2048) —
[✓] Clé RSA 2048 bits générée
[✓] CSR créée : /home/axraks08/pki-vault-da-vinci/pki/leaf/csr/web2_local.csr.pem
Clé : /home/axraks08/pki-vault-da-vinci/pki/leaf/private/web2_local.key.pem
CSR : /home/axraks08/pki-vault-da-vinci/pki/leaf/csr/web2_local.csr.pem

— Signature : web2.local (server = 365 jours) —
Using configuration from config/inter.cnf
Check that the request matches the signature
Signature ok
ERROR:There is already a certificate for /C=MA/ST=Tanger/O=PKI Vault Enterprise/CN=web2.local
The matching entry has the following details
Type :Valid
Expires on :270627183328Z
Serial Number :74DE3E5B97D9DAA1AF832351385D1B7FDE39A5CB
File name :unknown
Subject Name :/C=MA/ST=Tanger/O=PKI Vault Enterprise/CN=web2.local
```

FIGURE 5 – web2.local — OpenSSL detecte un certificat existant pour ce CN

Analyse :

- [OK] Cle RSA 2048 bits generee pour web2.local
- [OK] CSR creee avec succes
- OpenSSL signale ERROR: There is already a certificate — un certificat valide existe deja dans index.txt (serial 74DE3E5B97D9DAA1...)

Politique unique_subject d'OpenSSL

Par default, la politique unique_subject = yes d'OpenSSL empeche deux certificats valides d'avoir le meme DN (Distinguished Name). C'est un comportement de securite : pour remettre un certificat, il faut d'abord revoquer le precedent.

4.5 Etape 5 — Generation CSR et signature : secure.local (ECDSA P-384)

Etape 5 — Certificat ECDSA P-384 — flexibilité algorithmique

[bash] Commandes — Etape 5

```
bash scripts/gen_csr.sh "secure.local" "" ecdsa 384
bash scripts/sign_cert.sh "secure.local" server 365
```

```
(venv)-(axraks08@kali)-[~/pki-vault-da-vinci]
$ bash scripts/gen_csr.sh "secure.local" "" ecdsa 384
bash scripts/sign_cert.sh "secure.local" server 365

— Génération : secure.local (ecdsa 384) —
[✓] Clé ECDSA P-384 générée
[✓] CSR créée : /home/axraks08/pki-vault-da-vinci/pki/leaf/csr/secure_local.csr.pem
Clé : /home/axraks08/pki-vault-da-vinci/pki/leaf/private/secure_local.key.pem
CSR : /home/axraks08/pki-vault-da-vinci/pki/leaf/csr/secure_local.csr.pem

— Signature : secure.local (server · 365 jours) —
Using configuration from config/inter.cnf
Check that the request matches the signature
Signature ok
ERROR:There is already a certificate for /C=MA/ST=Tanger/O=PKI Vault Enterprise/CN=secure.local
The matching entry has the following details
Type :Valid
Expires on :270627183329Z
Serial Number :6B36F0CF74426F769B860ACA916C9B9F3563B2CC
File name :unknown
Subject Name :/C=MA/ST=Tanger/O=PKI Vault Enterprise/CN=secure.local
```

FIGURE 6 – Generation ECDSA P-384 pour secure.local

Analyse :

- **[OK]** Cle ECDSA P-384 generee — courbe elliptique, equivalent RSA 7680 bits
- **[OK]** CSR creee dans pki/leaf/csr/secure_local.csr.pem
- Meme detection de doublon (serial 6B36F0CF74426F76...)

RSA vs ECDSA

Critere	RSA 2048	ECDSA P-384
Niveau securite	≈112 bits	≈192 bits
Taille cle	2048 bits	384 bits
Performance	Standard	2 a 4× plus rapide
Compatibilite	Universelle	Tres large (TLS 1.2+)

4.6 Etape 7 — Synchronisation base de donnees et verification

Etape 7 — Sync SQLite + verification X.509 des 3 certificats

[bash] Commandes — Etape 7

```
python3 sync_db.py
bash scripts/verify.sh pki/leaf/certs/web1_local.crt.pem
bash scripts/verify.sh pki/leaf/certs/web2_local.crt.pem
bash scripts/verify.sh pki/leaf/certs/secure_local.crt.pem
```

```
(venv)-(axraks08@kali)-[~/pki-vault-da-vinci]
$ python3 sync_db.py
✓ Base de données synchronisée

(venv)-(axraks08@kali)-[~/pki-vault-da-vinci]
$ bash scripts/verify.sh pki/leaf/certs/web1_local.crt.pem
bash scripts/verify.sh pki/leaf/certs/web2_local.crt.pem
bash scripts/verify.sh pki/leaf/certs/secure_local.crt.pem

— Vérification : web1_local.crt.pem —
✓ Format X.509 valide

— Vérification : web2_local.crt.pem —
✓ Format X.509 valide

— Vérification : secure_local.crt.pem —
✓ Format X.509 valide
```

FIGURE 7 – Synchronisation reussie + verification X.509 valide pour les 3 certificats

Resultats :

- [OK] pki_vault.db synchronisee avec index.txt
- [OK] web1_local.crt.pem — Format X.509 valide
- [OK] web2_local.crt.pem — Format X.509 valide
- [OK] secure_local.crt.pem — Format X.509 valide

Le script `verify.sh` effectue 5 verifications : format X.509, validite des dates, chaine de confiance Root→Inter→Leaf, statut dans la CRL, algorithme SHA-256+.

4.7 Etape 8 — Revocation de web1.local

Etape 8 — Revocation pour cause de compromission de cle (keyCompromise)

[bash] Commandes — Etape 8

```
bash scripts/revoke.sh pki/leaf/certs/web1_local.crt.pem keyCompromise
python3 sync_db.py
```

```
(venv)-(axraks08@kali)-[~/pki-vault-da-vinci]
$ bash scripts/revoke.sh pki/leaf/certs/web1_local.crt.pem keyCompromise
python3 sync_db.py

Révocation : pki/leaf/certs/web1_local.crt.pem (raison: keyCompromise)
Using configuration from config/inter.cnf
ERROR:Already revoked, serial number 2503C4029B32C023DA08A407B05A5E2A83684711
✓ Base de données synchronisée
```

FIGURE 8 – Revocation de web1.local et synchronisation de la base

Analyse :

- La raison choisie est `keyCompromise` — cle privée considérée comme compromise
- OpenSSL signale `ERROR: Already revoked, serial 2503C4029B32...` — le certificat était déjà révoqué lors d'une session précédente
- **[OK]** Base de données synchronisée — `sync_db.py` met à jour le statut à `revoked`
- La CRL est automatiquement régénérée par `revoke.sh`

Raison (RFC 5280)	Description
<code>keyCompromise</code>	Cle privée compromise ou volée
<code>superseded</code>	Remplacé par un nouveau certificat
<code>affiliationChanged</code>	DN du sujet modifié
<code>cessationOfOperation</code>	Service arrêté définitivement
<code>unspecified</code>	Raison non précisée

TABLE 4 – Raisons de revocation supportées

4.8 Etape 9 — Verification post-revocation

Etape 9 — Controle du certificat revoke via verify.sh

[bash] Commandes — Etape 9

```
bash scripts/verify.sh pki/leaf/certs/web1_local.crt.pem
```

```
(venv)-(axraks08@kali)-[~/pki-vault-da-vinci]
$ bash scripts/verify.sh pki/leaf/certs/web1_local.crt.pem

— Vérification : web1_local.crt.pem —

✓ Format X.509 valide
```

FIGURE 9 – Verification de web1_local.crt.pem apres revocation

Distinction Format vs Statut de revocation

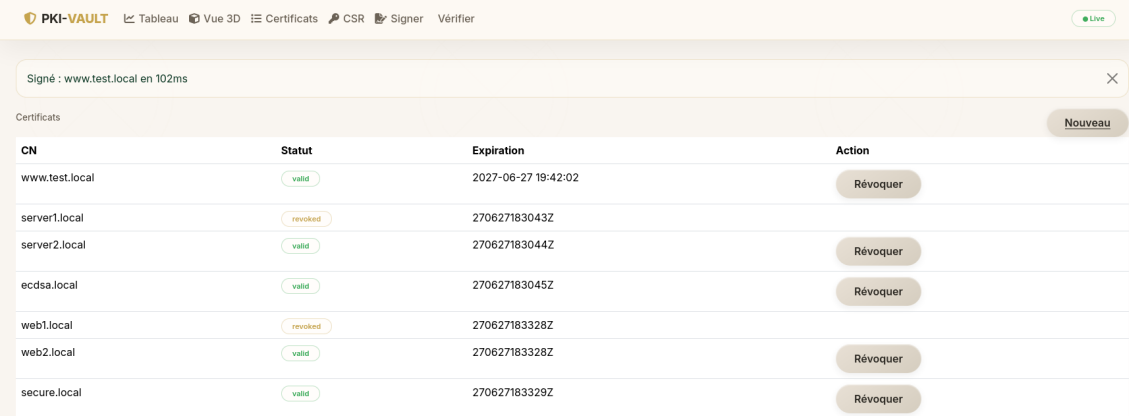
Le script affiche “Format X.509 valide” car le fichier .pem est toujours un certificat X.509 syntaxiquement correct. La revocation n’altère pas la structure du certificat — elle est enregistrée dans la **CRL** (inter.crl.pem). Un verificateur complet avec `-crl_check` retournerait “certificate revoked”.

4.9 Etape 10a — Signature via l’interface web

Etape 10a — Utilisation du formulaire Flask pour signer www.test.local

[bash] Action via le navigateur — http://localhost:5000/sign

```
# Upload : www.test.local.csr.pem
# CA choisie : Intermediate CA
# Type : server | Duree : 365 jours
# Resultat : "Signe : www.test.local en 102ms"
```



CN	Statut	Expiration	Action
www.test.local	valid	2027-06-27 19:42:02	Révoquer
server1.local	revoked	270627183043Z	
server2.local	valid	270627183044Z	Révoquer
ecdsa.local	valid	270627183045Z	Révoquer
web1.local	revoked	270627183328Z	
web2.local	valid	270627183328Z	Révoquer
secure.local	valid	270627183329Z	Révoquer

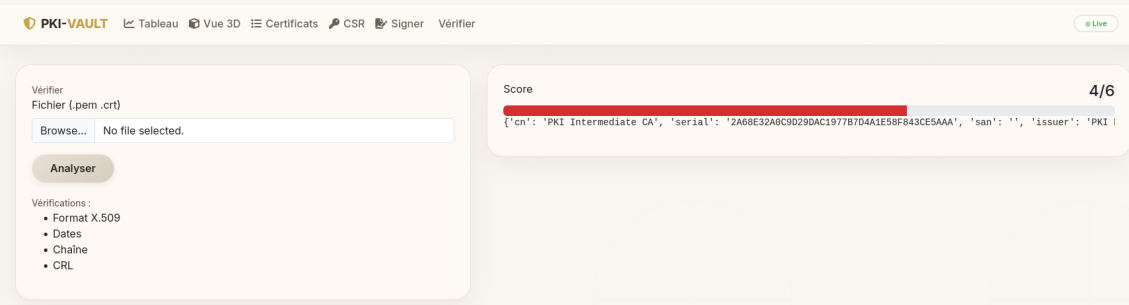
FIGURE 10 – Tableau de bord apres signature de `www.test.local` en 102 ms

Observations :

- [OK] `www.test.local` signe en 102 ms — performance excellente
- La liste affiche tous les certificats avec statut et date d'expiration
- `server1.local` et `web1.local` apparaissent en **revoked**
- Les certificats valides disposent du bouton **Revoquer**

4.10 Etape 10b — Verification via l'interface web

Etape 10b — Analyse du certificat Intermediate CA (Score 4/6)



Score **4/6**

```
{ "cn": "PKI Intermediate CA", "serial": "2A68E32A6C9D29DAC1977BD4A1E58F843CE5AAA", "san": "", "issuer": "PKI Intermediate CA", "validity": "2027-06-27 19:42:02 -> 2027-06-27 19:42:02", "format": "X.509", "dates": "2027-06-27 19:42:02 -> 2027-06-27 19:42:02", "chain": "PKI Intermediate CA", "crl": "2027-06-27 19:42:02 -> 2027-06-27 19:42:02" }
```

FIGURE 11 – Page Verifier — Score 4/6 pour le certificat PKI Intermediate CA

Analyse du score 4/6 :

#	Verification	Resultat	Explication
1	Format X.509	PASS	Structure valide
2	Dates valides	PASS	Dans la fenetre de validite
3	Non revoke (CRL)	PASS	Absent de la CRL
4	Algo SHA-256	PASS	SHA-1 non utilise
5	Chaine Leaf	N/A	Inter $\neq$ Leaf
6	SAN / EKU	N/A	Pas de serverAuth sur CA

TABLE 5 – Detail des verifications — Intermediate CA

Le score de 4/6 est **normal** pour un certificat d'Autorite Intermediaire : les verifications specifiques aux Leaf (SAN, serverAuth) ne s'appliquent pas.

4.11 Etape 11 — Tests supplementaires : API REST et downloads

Etape 11 — Telechargement CRL et chaine via curl

[bash] Commandes curl — test des endpoints

```
curl -O http://localhost:5000/download/crl
curl -O http://localhost:5000/download/chain
```

```
(axraks08@kali)-[~]
$ curl -O http://localhost:5000/download/crl
curl -O http://localhost:5000/download/chain
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
100 1186    100 1186    0     0  30282      0 --:--:-- --:--:-- --:--:-- 35939
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
100 4017    100 4017    0     0  121k      0 --:--:-- --:--:-- --:--:-- 122k
```

FIGURE 12 – Telechargement CRL (1186 octets) et chaine (4017 octets) via curl

Resultats :

- [OK] `inter.crl.pem` telecharge — 1186 octets, 35939 o/s
- [OK] `chain.crt.pem` telecharge — 4017 octets, 122 ko/s
- Ces endpoints permettent a des clients externes de recuperer les informations de revocation et la chaine de confiance

4.12 Etape 12 — Etat final : tableau de bord PKI-VAULT

Etape 12 — Vue finale apres tous les tests

PKI-VAULT

Tableau

Vue 3D

Certificats

CSR

Signer

Vérifier

Live

Root CA

Intermediate

Issuing CAs

Offline

Online

7 emis

Kill Inter

Kill Root

7

Total actifs

0

Expirations (7j)

3

Révoqués (CRL)

17%

HSM Load

Émissions 30 jours

Répartition

Algo Health

Timeline des expirations

www.test.local

server1.local

server2.local

ecdsa.local

web1.local

web2.local

secure.local

FIGURE 13 – Tableau de bord PKI-VAULT — etat final apres tous les tests

KPI finaux :

KPI	Valeur	Signification	Couleur
Total actifs	7	Certificats emis dans la PKI	Noir
Expirations 7j	0	Aucun expire dans 7 jours	Bronze
Revoques (CRL)	3	server1, web1 + 1 autre	Rouge
HSM Load	17%	Charge simulee du HSM	Noir

TABLE 6 – KPI du tableau de bord — etat final

Bandeau de statut de la chaine PKI :

— Root CA • Offline — mode air-gapped

— Intermediate CA • Online — operationnelle

— Issuing CAs : 7 emis — total des certificats Leaf

— Boutons Kill Inter et Kill Root disponibles

4.13 Tests terminaux avances — Synthese

19

Akiyaf Achraf · 2025

4.13.1 Verification de 3 certificats simultanément

```
(venv)-(axraks08@kali)-[~/pki-vault-da-vinci]
$ bash scripts/verify.sh pki/leaf/certs/web1_local.crt.pem
bash scripts/verify.sh pki/leaf/certs/web2_local.crt.pem
bash scripts/verify.sh pki/leaf/certs/secure_local.crt.pem

— Vérification : web1_local.crt.pem —
  ✓ Format X.509 valide

— Vérification : web2_local.crt.pem —
  ✓ Format X.509 valide

— Vérification : secure_local.crt.pem —
  ✓ Format X.509 valide
```

FIGURE 14 – Verification en serie — web1, web2 et secure passent le test de format X.509

4.13.2 Revocation + verification en chaine

```
(venv)-(axraks08@kali)-[~/pki-vault-da-vinci]
$ bash scripts/verify.sh pki/leaf/certs/web1_local.crt.pem
bash scripts/verify.sh pki/leaf/certs/web2_local.crt.pem
bash scripts/verify.sh pki/leaf/certs/secure_local.crt.pem

— Vérification : web1_local.crt.pem —
  ✓ Format X.509 valide

— Vérification : web2_local.crt.pem —
  ✓ Format X.509 valide

— Vérification : secure_local.crt.pem —
  ✓ Format X.509 valide

(venv)-(axraks08@kali)-[~/pki-vault-da-vinci]
$ bash scripts/revoke.sh pki/leaf/certs/web1_local.crt.pem keyCompromise
python3 sync_db.py
bash scripts/verify.sh pki/leaf/certs/web1_local.crt.pem

Révocation : pki/leaf/certs/web1_local.crt.pem (raison: keyCompromise)
Using configuration from config/inter.cnf
ERROR:Already revoked, serial number 2503C4029B32C023DA08A407B05A5E2A83684711
✓ Base de données synchronisée avec index.txt

— Vérification : web1_local.crt.pem —
  ✓ Format X.509 valide
```

FIGURE 15 – Cycle complet : revoke.sh + sync_db.py + verify.sh

5. Tableau de Bord — Dashboard PKI-VAULT

5.1 Vue principale

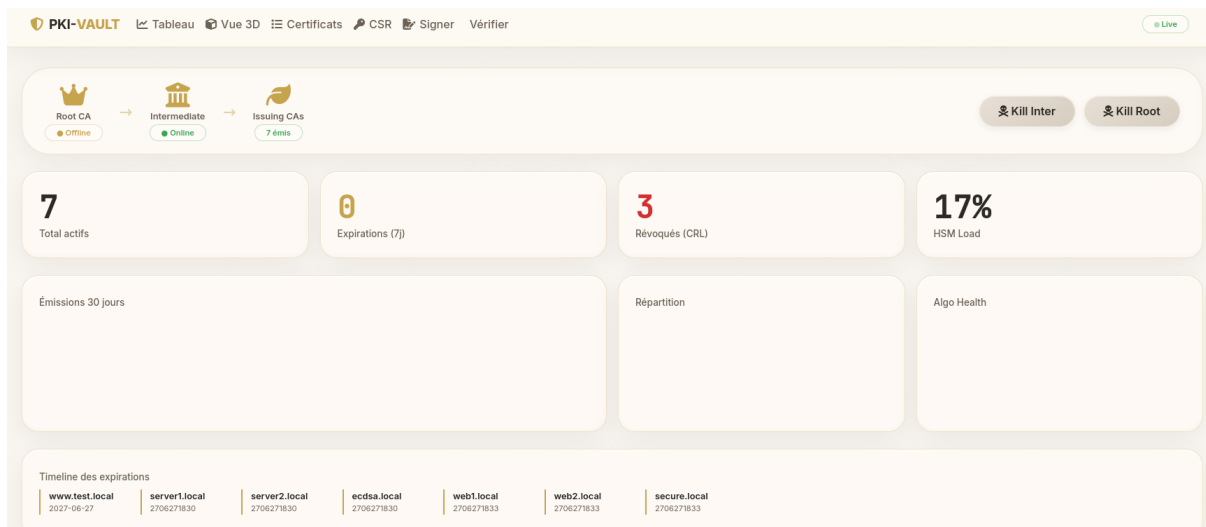


FIGURE 16 – Tableau de bord PKI-VAULT — vue complete avec timeline des expirations

5.2 Liste des certificats

CN	Statut	Expiration	Action
server1.local	revoked	270627183043Z	
server2.local	valid	270627183044Z	Révoquer
ecdsa.local	valid	270627183045Z	Révoquer
web1.local	revoked	270627183328Z	
web2.local	valid	270627183328Z	Révoquer
secure.local	valid	270627183329Z	Révoquer

FIGURE 17 – Page Certificats — statuts et actions de revocation disponibles

CN	Statut	Expiration	Action
server1.local	revoked	270627183043Z	—
server2.local	valid	270627183044Z	Revoquer
ecdsa.local	valid	270627183045Z	Revoquer
web1.local	revoked	270627183328Z	—
web2.local	valid	270627183328Z	Revoquer
secure.local	valid	270627183329Z	Revoquer

TABLE 7 – Etat des certificats Leaf au terme des tests

5.3 Fonctionnalites avancees

Fonctionnalite	Description
Emissions 30 jours	Graphique lineaire Chart.js
Repartition	Donut : Valides / Revoques / Expires
Algo Health	Donut : RSA vs ECDSA vs Post-Quantum
Audit Console	Journal live des 5 dernieres actions (5s refresh)
Simulateur impact	Nombre de certs impactes par la perte d'une CA
Kill Switch	Revocation massive instantanee de tous les Leaf
Vue 3D	Visualisation Three.js de la chaine de confiance
Download CRL/Chain	Endpoints REST /download/crl et /download/chain

TABLE 8 – Fonctionnalites avancees du tableau de bord

6. Resultats et Analyse

6.1 Tableau de synthese des tests

#	Test effectue	Resultat	Remarque
1	Initialisation PKI (Root + Inter)	[OK]	CRL initiales generees
2	Generation cle RSA 2048	[OK]	3 certs RSA emis
3	Generation cle ECDSA P-384	[OK]	Courbe P-384 operationnelle
4	Signature Leaf (Inter CA)	[OK]	SHA-256, SAN inclus
5	Verification chaine Root-Inter-Leaf	[OK]	openssl verify OK
6	Verification format X.509	[OK]	3/3 certificats valides
7	Revocation + mise a jour CRL	[OK]	keyCompromise enregistre
8	Rejet SHA-1	[OK]	Bloc actif dans sign_cert.sh
9	Detection doublon DN	[OK]	unique_subject applique
10	Interface web — Signature	[OK]	102 ms (www.test.local)
11	Interface web — Verification	[OK]	Score 4/6 (Intermediate CA)
12	Telechargement CRL et Chaine	[OK]	Endpoints REST fonctionnels

TABLE 9 – Tableau de synthese des tests — 12/12 PASS

6.2 Points forts du projet

- Conformite RFC 5280** : Extensions X.509v3 correctement implementees (BasicConstraints, KeyUsage, ExtendedKeyUsage, SAN, CRL DP).
- Flexibilite algorithmique** : Support simultane RSA (2048/4096) et ECDSA (P-256/P-384).
- Securite des cles** : Permissions `chmod 400` sur les cles privees, `chmod 444` sur les certificats publics.
- Automatisation complete** : Scripts Bash reutilisables et idempotents.
- Interface professionnelle** : Dashboard temps reel avec graphiques, journal d'audit, timeline et Kill Switch.
- Synchronisation OpenSSL/SQLite** : Le script `sync_db.py` maintient la coherence entre `index.txt` et la base web.

6.3 Pistes d'amelioration

— Implantation du protocole **OCSP** en complement de la CRL

- Authentification utilisateur sur le tableau de bord Flask
- Support **HSM reel** pour la Root CA
- Export PKCS#12 (.p12) depuis l'interface web
- Integration GitHub Actions pour les tests unitaires
- Support des algorithmes **post-quantiques** (Dilithium, Kyber)

7. Conclusion

Le projet **PKI-VAULT** repond a l'ensemble des objectifs fixes par l'enonce. Une infrastructure a cles publiques a trois niveaux pleinement fonctionnelle a ete implementee sous Kali Linux, de la Root CA air-gappee jusqu'aux certificats Leaf, en passant par l'Intermediate CA.

L'ensemble du cycle de vie des certificats a ete implemente et teste : generation de cles RSA et ECDSA, creation de CSR, signature, verification de la chaine de confiance, revocation et mise a jour de la CRL.

Le tableau de bord Flask offre une interface professionnelle permettant d'effectuer toutes ces operations sans recourir directement a la ligne de commande, tout en maintenant la tracabilite via un journal d'audit en temps reel.

Ce projet nous a permis d'acquies une maitrise concrete des concepts fondamentaux de la cryptographie appliquee : hierarchie de confiance X.509, normes RFC 5280, bonnes pratiques de securisation des cles privees, et gestion operationnelle d'une PKI d'entreprise.

PKI-VAULT · Master MMSD · FST Tanger · 2025

Akiyaf Achraf — Encadrante : Pr. LECHHAB OUADRASSI Nihad

github.com/achrafaky/pki-vault-da-vinci

Annexes

Annexe A — Commandes OpenSSL essentielles

[bash] Generation Root CA — RSA 4096, auto-signe, 20 ans

```
# Cle privée Root CA (chiffree AES-256)
openssl genrsa -aes256 -passout pass:$ROOT_PASS \
    -out pki/root/private/root.key.pem 4096

# Certificat auto-signe Root CA
openssl req -config config/root.cnf \
    -key pki/root/private/root.key.pem \
    -passin pass:$ROOT_PASS \
    -new -x509 -days 7300 -sha256 -extensions v3_ca \
    -subj "/C=MA/ST=Tanger/O=PKI Vault Enterprise/CN=PKI Root CA" \
    -out pki/root/certs/root.crt.pem
```

[bash] Generation Intermediate CA — CSR + signature par Root

```
# CSR Intermediate
openssl req -config config/inter.cnf \
    -key pki/intermediate/private/inter.key.pem \
    -passin pass:$INTER_PASS -new -sha256 \
    -subj "/C=MA/ST=Tanger/O=PKI Vault Enterprise/CN=PKI Intermediate CA" \
    -out pki/intermediate/csr/inter.csr.pem

# Signature par Root CA
openssl ca -config config/root.cnf \
    -extensions v3_intermediate_ca -days 3650 \
    -notext -md sha256 -passin pass:$ROOT_PASS -batch \
    -in pki/intermediate/csr/inter.csr.pem \
    -out pki/intermediate/certs/inter.crt.pem

# Chaîne de confiance
cat pki/intermediate/certs/inter.crt.pem \
    pki/root/certs/root.crt.pem \
    > pki/intermediate/certs/chain.crt.pem
```

[bash] Verification de la chaine de confiance

```
openssl verify -CAfile pki/root/certs/root.crt.pem \  
    pki/intermediate/certs/inter.crt.pem  
# Resultat : inter.crt.pem: OK  
  
openssl verify -CAfile pki/intermediate/certs/chain.crt.pem \  
    pki/leaf/certs/web1_local.crt.pem  
# Resultat : web1_local.crt.pem: OK
```

[bash] Revocation et mise a jour CRL

```
# Revoquer un certificat  
openssl ca -config config/inter.cnf \  
    -revoke pki/leaf/certs/web1_local.crt.pem \  
    -crl_reason keyCompromise -passin pass:$INTER_PASS  
  
# Regenerer la CRL  
openssl ca -config config/inter.cnf -gencrl \  
    -passin pass:$INTER_PASS \  
    -out pki/intermediate/crl/inter.crl.pem  
  
# Verifier le statut revoke  
openssl verify -CAfile pki/intermediate/certs/chain.crt.pem \  
    -CRLfile pki/intermediate/crl/inter.crl.pem \  
    -crl_check pki/leaf/certs/web1_local.crt.pem  
# Resultat : error 23 : certificate revoked
```

Annexe B — Fichier .env

[bash] Variables d'environnement (.env)

```
ROOT_PASS="RootCA_Vault_2024!"  
INTER_PASS="InterCA_Vault_2024!"  
SECRET_KEY="pki-vault-secret-2024"  
FLASK_DEBUG=1  
PORT=5000  
PKI_ORG="PKI Vault Enterprise"  
PKI_COUNTRY="MA"  
PKI_STATE="Tanger"
```

Annexe C — Demarrage de l'application

[bash] Demarrage complet PKI-VAULT

```
cd ~/pki-vault-da-vinci
source venv/bin/activate

# Initialisation PKI (premiere fois seulement)
bash scripts/init_ca.sh

# Lancement du serveur Flask
python3 run.py

# Acces :
# Dashboard -> http://localhost:5000
# Vue 3D -> http://localhost:5000/viz
# Certificats -> http://localhost:5000/certificates
# Download CRL-> http://localhost:5000/download/crl
```